

Pull Request & Branch Workflow Guidelines

- To maintain a clean, stable, and highly collaborative codebase within the Anchor repository, all development work must adhere to the following structured guidelines. These procedures are designed to streamline development, enhance code quality through systematic reviews, and ensure seamless integration of new features and bug fixes.
- **Branching Strategy** We employ a robust, structured branch workflow to facilitate smooth collaboration, isolate development efforts, and maintain the integrity of our production environment. Our core branching model is as follows:

main → **dev** → **feature/<member-name-or-task>**

- **main Branch:** This is our cornerstone branch, representing the absolute production-ready, stable codebase. Only thoroughly tested and approved code from the **dev** branch is ever merged into **main**. Direct commits or merges into **main** are strictly prohibited.
- **dev Branch:** The **dev** branch serves as our primary integration branch. All approved and reviewed feature work is merged into **dev** before being prepared for a release. This branch should always reflect the latest integrated, pre-production state of the application. It undergoes continuous testing and stabilization.
- **feature/* Branches:** These are individual, short-lived branches created for specific development tasks, features, or bug fixes. Each team member or distinct task should have its own feature branch, ensuring that development efforts are isolated and do not interfere with other ongoing work.

Examples of Feature Branch Naming Conventions:

- **feature/meghana-auth** (for an authentication module developed by Meghana)

Development Rules

Adherence to these fundamental development rules is critical for maintaining code quality, preventing conflicts, and ensuring efficient team collaboration:

- **Always Branch from dev:** All new feature branches, bug fix branches, and any other development branches **must** originate from the latest **dev** branch. **Never branch directly from main.** This ensures that your work is based on the most up-to-date integrated codebase.
- **Never Push Directly to main or dev:** Direct pushes to either the **main** or **dev** branches are strictly forbidden. All changes, no matter how minor, must go through the

Pull Request (PR) and code review process. This policy is paramount for maintaining code quality and stability.

- **Each Change Requires a Pull Request (PR) and Code Review:** Every modification, addition, or deletion of code must be submitted via a Pull Request. This allows other team members to review the changes, provide feedback, identify potential issues, and ensure adherence to coding standards before the code is integrated.
- **Clear and Meaningful Commit Messages:** Commit messages are vital for tracking changes and understanding code history. They should be concise yet descriptive, following conventional commit guidelines where applicable.
 - **Good Examples:**
 - `fix: handle null user case in profile display`
 - `docs: update API documentation for new endpoint`
 - `chore: update third-party library dependencies`

Pull Request Workflow

The Pull Request (PR) workflow is the cornerstone of our collaborative development process, ensuring that all code changes are reviewed, tested, and approved before integration.

1. Create Your Feature Branch:

- First, ensure your local `dev` branch is up-to-date:
`git checkout dev`
- `git pull origin dev`
- Then, create your new feature branch:
`git checkout -b feature/<your-feature-name>`
- Choose a descriptive name for your feature branch that clearly indicates its purpose.

2. Make Changes and Test Locally:

- Implement your feature, fix your bug, or make your desired changes within your new feature branch.
- Thoroughly test your changes locally to ensure they function as expected and do not introduce regressions. Write new tests or update existing ones if applicable.

3. Commit and Push Your Changes:

- Stage all your relevant changes:
`git add .`
- Commit your changes with a clear and meaningful message:
`git commit -m \"feat: implement user registration form\"`
- Push your feature branch to the remote repository:
`git push origin feature/<your-feature-name>`

4. Open a Pull Request (PR):

- Navigate to your Git hosting platform (e.g., GitHub, GitLab, Bitbucket).
- Create a new Pull Request.
- **Base Branch:** Set the **base branch** (the branch you want to merge into) to `dev`.
- **Compare Branch:** Set the **compare branch** (your feature branch) to

feature/<your-feature-name>.

- **Title Your PR Clearly:** The PR title should be concise and accurately describe the overall purpose of your changes.
 - **Example:** Add onboarding screen for iOS or Fix bug with password reset link
- **Describe Your Changes in the PR Body:** Provide a brief, yet comprehensive, description of your changes. Include:
 - What problem does this PR solve?
 - What changes were made?

5. Code Review:

- After opening the PR, request reviews from at least one, and ideally two, designated team members.
- **Wait for Approval:** Your PR must receive at least one approval from a designated reviewer before it can be merged. More complex changes may require multiple approvals.
- **Address Review Comments Promptly:** Engage with reviewers, clarify questions, and implement suggested changes. Make additional commits to your feature branch to address feedback.
- Reviewers will check for:
 - Correctness and functionality
 - Adherence to coding standards and best practices
 - Code readability and maintainability
 - Test coverage
 - Potential side effects or regressions

6. Merge Your PR:

- Once your PR has received the required approvals and all comments have been addressed, you may merge your feature branch into the `dev` branch.
- **Important:** Only the release manager or designated individual will merge `dev` into `main` after all features for a release cycle are tested and stable.

Important Notes and Best Practices

To ensure a smooth, high-quality development process, please keep the following in mind:

- **Do Not Merge Your Own PR Without Review:** It is a critical policy that you do not merge your own Pull Request unless explicitly approved by the team lead or release manager for specific, low-risk scenarios. Self-merging bypasses the essential code review process.
- **Avoid Large PRs:** Aim for smaller, focused commits and Pull Requests. Large PRs are difficult to review effectively, increase the likelihood of introducing bugs, and slow down the review process. Break down complex features into smaller, manageable chunks.
- **Keep Your Branch Up-to-Date with `dev`:** Before merging your PR, it's crucial to ensure your feature branch is up-to-date with the latest changes from the `dev` branch. This helps to resolve potential merge conflicts early and ensures your code integrates cleanly.

- To update your branch:
git fetch origin
- git merge origin/dev
- Resolve any merge conflicts that arise during this process.

Example PR Flow in Action

Let's illustrate the workflow with a practical scenario:

1. **Meghana creates `feature/meghana-auth`:** She branches from `dev`, implements an authentication module, and tests it.
2. **Pushes Changes & Opens PR:** Meghana pushes her changes to her remote feature branch and then opens a Pull Request targeting the `dev` branch.
3. **Pooja Reviews & Approves:** Pooja reviews Meghana's code, provides feedback, which Meghana addresses. Once satisfied, Pooja approves the PR.
4. **PR Merged:** With approval, Meghana (or a designated team member) merges `feature/meghana-auth` into `dev`. The `dev` branch now includes the new authentication module.
5. **Shiva Prepares for Release:** Once `dev` is stable and all features for the current release are integrated and tested, Rahul (as the designated release manager) prepares for the release.
6. **Opens PR `dev` → `main`:** opens a final Pull Request from `dev` into `main`.
7. **Final Review & Merge:** The team performs a final high-level review of the `dev` to `main` PR. Once approved, Rahul merges `dev` into `main`.
8. **Deployed:** The `main` branch now reflects the new production-ready code, which is then deployed to users.